Security Audit Report

Kolekto — POS · Rebrand + Security Delta

Project	**Kolekto — POS**
Date	2026-05-08
Version	V2026.05.08-001
Auditor	Claude Code (automated)
Branch	feat/rebrand-kolekto-v1
Scope	Auth & Access Control · Privacy Boundaries · API Security · Data Integrity · Production Hygiene
Delta from	2026-05-07-v1 (6 findings remediated, 3 new findings)

RISK SCORE **62/100** ● RED **+10 vs v1**

CRITICAL	HIGH	MEDIUM	LOW	INFO
1	2	2	3	3

Executive Summary

This audit covers the Kolekto rebrand (feat/rebrand-kolekto-v1) and documents the security posture delta since the previous audit (2026-05-07-v1). Six findings from the prior audit were fully remediated: rate limiting (slowapi), audit logging, JWT token expiry, CSP/HSTS headers, SQLAlchemy echo, and security headers in next.config.ts. Three new findings were identified: two IDOR vulnerabilities on cashier sessions and individual sales (no ownership checks), and a weak supervisor authorization scope on the void-sale action. The overall risk score improved by +10 points (52 → 62). Highest priority is the cashier session IDOR (CRITICAL). The rebrand itself introduced no new security vulnerabilities — all changes were visual/string.

Remediated Since v1 (2026-05-07)

The following findings from the 2026-05-07-v1 audit have been fully resolved:

✓ F001-v1: Rate limiting — FIXED (slowapi, login 10/min)
✓ F002-v1: Audit logs absent — FIXED (all mutations logged)
✓ F004-v1: JWT 8h expiry — FIXED (60 minutes)
✓ F006-v1: CSP/HSTS missing in Caddy — FIXED
✓ F008-v1: SQLAlchemy echo=True — FIXED (echo=False always)
✓ F010-v1: No security headers in next.config.ts — FIXED

Compliance Framework Scores

Scored as baseline security posture indicators. HIPAA and CMMC are not in regulatory scope for this personal project but are included for completeness.

Framework	Score	%	Status
SOC2 TSC	14.5/28	52%	■ NEEDS WORK
HIPAA	8.5/18	47%	✗ FAILING
CMMC L2	12.0/22	55%	■ NEEDS WORK
ISO 27001:2022	11.0/24	46%	✗ FAILING

Control Detail by Framework

SOC2 TSC

Control	Note	Score
CC6.1	IDOR findings reduce access control completeness	Partial
CC6.2	JWT authentication complete, bcrypt passwords	Complete
CC6.3	RBAC present but IDOR bypasses authorization	Partial
CC6.4	Audit logging complete on all mutations	Complete
CC6.5	60-min token expiry implemented	Complete
CC6.6	Rate limiting on auth; missing on gift cards	Partial
CC6.7	Response schemas safe; IDOR reduces data access control	Partial
CC6.8	N/A for this project type	Partial
CC6.9	TLS at Caddy layer; HSTS misconfigured	Partial
CC7.1	Structlog structured logging throughout	Complete
CC7.2	Logs exist; no automated alerting on auth failures	Partial
CC7.3	No documented incident response plan	Absent

CC7.4	No incident tracking or escalation procedures	Absent
CC7.5	No anomaly detection or SIEM integration	Absent
CC7.6	Security audit conducted; not fully automated	Partial
CC8.1	Version control, migrations, CI/CD present	Complete
CC8.2	RBAC provisioning exists; no formal review process	Partial
CC8.3	Audit done; no automated SAST/DAST in pipeline	Partial
CC8.4	Code reviewed informally; no formal review gates	Partial
CC9.1	Some risk documentation; no formal risk register	Partial
CC9.2	Limited vendor dependencies; no formal assessment	Partial
CC9.3	No documented business continuity plan	Absent
CC9.4	N/A for this project scope	Partial
A1.1	Health check endpoint; no HA configuration	Partial
A1.2	No performance monitoring or SLA tracking	Absent
A1.3	Docker restart policies; no formal RTO/RPO	Partial
A1.4	Backup config exists in env vars; not verified active	Absent
CC6.1b	Full Pydantic validation on all endpoints	Complete

HIPAA

Control	Note	Score
§164.308(a)(1)	Partial risk analysis in audit; no formal risk management program	Partial
§164.308(a)(1)(ii)(D)	Security event logging present; no automated alerting	Partial
§164.308(a)(2)	No designated security officer	Absent
§164.308(a)(3)	RBAC workforce access controls; no clearance process	Partial
§164.308(a)(4)	Access management via RBAC; IDOR weakens this	Partial
§164.308(a)(5)	No security awareness training program	Absent
§164.308(a)(5)(ii)(C)	Login monitoring via audit logs; no automated alerts	Partial
§164.308(a)(5)(ii)(D)	Password policies exist; default password risk	Partial
§164.308(a)(6)	No documented security incident response procedures	Absent
§164.308(a)(7)	No contingency planning documented	Absent
§164.308(a)(8)	Security audit conducted; not on regular schedule	Partial
§164.312(a)(1)	Access control implemented; IDOR vulnerabilities present	Partial
§164.312(a)(2)(i)	Unique user IDs enforced	Complete
§164.312(a)(2)(iii)	60-min token expiry; no server-side invalidation on logout	Partial
§164.312(b)	Comprehensive audit logging on all mutations	Complete
§164.312(c)	Pydantic validation; no hash-based integrity verification	Partial
§164.312(d)	JWT authentication implemented correctly	Complete
§164.312(e)	TLS at proxy layer; HSTS misconfigured	Partial

CMMC L2

Control	Note	Score
AC.1.001	Access control policy partially implemented; IDOR findings	Partial
AC.1.002	Access limited to authorized users; IDOR bypasses	Partial
AC.2.006	Login rate limiting implemented	Complete
AC.2.007	Privileged account controls via RBAC; no audit review	Partial
AC.2.147	Authorization enforcement partial; tenant isolation absent	Partial
AU.2.041	Audit logs created for all mutations	Complete
AU.2.042	User identifiability in all audit records	Complete
AU.3.045	No automated audit log review or analysis	Absent
CM.2.061	Docker Compose baseline; no hardened baseline docs	Partial
CM.2.064	Config managed via env vars; no formal governance	Partial
CM.3.165	Partial; .env.test still in git	Partial
IA.1.076	All users identified and authenticated via JWT	Complete
IA.1.077	Password hashing (bcrypt); weak default admin password	Partial
IA.3.083	No MFA for any account	Absent
IR.2.092	No incident response plan	Absent
IR.2.093	No incident tracking or documentation	Absent
RA.2.137	Security audit conducted; not on defined schedule	Partial
RA.3.144	Manual vulnerability assessment only	Partial
SC.3.177	TLS at proxy; HSTS ineffective on HTTP-only config	Partial
SC.3.178	Full Pydantic input validation; CORS locked	Complete
SC.3.181	Security headers in both next.config.ts and Caddyfile	Complete
SI.3.171	SSRF fix applied in v1 (logo_url domain validation)	Partial

ISO 27001:2022

Control	Note	Score
A.5.15	Access control partially implemented; IDOR findings	Partial
A.5.16	User identity management complete	Complete
A.5.17	Authentication implemented; no MFA for privileged accounts	Partial
A.5.18	Access rights enforced by RBAC; IDOR bypasses	Partial
A.5.19	Limited external vendor risk; no formal assessment	Partial
A.6.1	No designated information security roles	Absent
A.6.3	No security awareness program	Absent
A.7.1	N/A cloud-hosted; Docker isolation present	Partial
A.8.2	Privileged access via RBAC; no formal review	Partial
A.8.3	Information access restriction partial; IDOR	Partial
A.8.4	Source code managed; no formal review gates	Partial
A.8.5	Authentication implemented; default password risk	Partial
A.8.6	No capacity management monitoring	Absent
A.8.7	Input validation complete; no anti-malware specific	Partial

A.8.9	Config via env vars; some hardening gaps	**Partial**
A.8.11	Schemas prevent PII leakage; no formal masking	**Partial**
A.8.12	Structured logging protects PII; no DLP	**Partial**
A.8.16	Audit logs present; no automated monitoring	**Partial**
A.8.20	CORS locked; security headers implemented	**Partial**
A.8.21	Service security partial; HSTS misconfigured	**Partial**
A.8.22	No network segmentation between services	**Absent**
A.8.24	TLS at proxy; no DB encryption at rest	**Partial**
A.8.25	Secure dev practices followed; no formal SDLC	**Partial**
A.8.29	Security testing conducted via this audit	**Complete**

Security Findings

■ CRITICAL

[F001] IDOR on cashier session access — any authenticated user can read any session

What: GET /api/v1/sales/sessions/{session_id} fetches any cashier session without verifying the requesting user owns it

Location: `backend/app/routers/sales.py:103-118`

Risk: Any cashier (or any other role) can read financial reconciliation data (starting cash, closing amounts, discrepancy) from another cashier's session by guessing or iterating UUIDs. UUID brute-force is impractical, but authenticated insiders can access sensitive data of colleagues.

Fix: In get_session_by_id: load the session, then check `if cashier_session.cashier_id != current_user.id and current_user.role not in ('admin', 'supervisor'): raise HTTPException(403)`. Add equivalent check in the service layer.

SOC2: CC6.1, CC6.3, CC6.7 **HIPAA:** 164.312(a)(1), 164.312(a)(2)(i) **CMMC:** AC.1.001, AC.1.002, AC.2.147 **ISO27001:** A.5.15, A.5.18, A.8.3

■ HIGH

[F002] IDOR on individual sale access — any authenticated user can read any sale

What: GET /api/v1/sales/{sale_id} returns full sale details (customer, items, payment method, amounts) without ownership or role check

Location: `backend/app/routers/sales.py:198-206`

Risk: A cashier can read sales created by other cashiers, accessing customer data, payment references, and business intelligence. An attacker who compromises a low-privilege account can enumerate all sales.

Fix: Add ownership check: `if sale.cashier_id != current_user.id and current_user.role not in ('admin', 'supervisor'): raise HTTPException(403)`. Apply same pattern to void_sale and any other sale-by-ID endpoints.

SOC2: CC6.1, CC6.3 **HIPAA:** 164.312(a)(1) **CMMC:** AC.1.001, AC.2.147 **ISO27001:** A.5.15, A.8.3

[F003] Weak supervisor authorization on void_sale — no location/cashier scope enforcement

What: POST /api/v1/sales/{sale_id}/void only checks role=supervisor; does not verify the supervisor manages the cashier who created the sale

Location: `backend/app/routers/sales.py:209-221`

Risk: A supervisor at one location can void sales created by cashiers at any other location, enabling unauthorized financial manipulation across the business.

Fix: In void_sale service: check that `sale.cashier_id == current_user.id OR current_user.role == 'admin'`. For multi-location deployments, additionally scope by location_id or supervisor assignment.

SOC2: CC6.3, CC6.4 **HIPAA:** 164.308(a)(4) **CMMC:** AC.2.007 **ISO27001:** A.5.18, A.8.2

■ MEDIUM

[F004] Default admin password 'Admin123!' — startup warning but no enforcement

What: ADMIN_INITIAL_PASSWORD defaults to 'Admin123!' in config.py. A warning is logged at startup but nothing blocks the app from running with this default.

Location: `backend/app/config.py:49, backend/app/main.py:30–35`

Risk: If ADMIN_INITIAL_PASSWORD is not overridden in the .env file (e.g., fresh install, copied config), the admin account has a publicly known password attackable by credential stuffing.

Fix: Enforce non-default password at startup: if settings.admin_initial_password == DEFAULT, raise RuntimeError in production (ENV=production). Alternatively, generate a random password and display it once on first boot, then require reset.

SOC2: CC6.2 **HIPAA:** 164.308(a)(5)(ii)(D) **CMMC:** IA.1.077 **ISO27001:** A.9.4.3

[F005] Tenant isolation absent at DB query level

What: All service queries do not filter by tenant_id. Models have tenant_id with a hardcoded default UUID but queries never enforce tenant boundaries.

Location: `backend/app/services/*.py (all service files)`

Risk: If multi-tenant deployment is ever attempted, all data from all tenants is accessible to any authenticated user. Also creates IDOR risk if tenant isolation is added inconsistently later.

Fix: Add tenant_id to all WHERE clauses as a baseline. Create a TenantContext dependency that injects tenant_id from the authenticated user. Fail closed: 404 if entity.tenant_id != current_user.tenant_id.

SOC2: CC6.1, CC6.3 **HIPAA:** 164.312(a)(1) **CMMC:** AC.2.147 **ISO27001:** A.9.1.2

■ LOW

[F006] backend/.env.test committed to git — test credentials in version control

What: backend/.env.test is tracked by git and contains DATABASE_URL, SECRET_KEY (test values). Added to .gitignore but still tracked.

Location: `backend/.env.test`

Risk: Although credentials are test-only (not production), this normalizes credential storage in git. Developers may base production configs on this pattern. SECRET_KEY format is exposed.

Fix: Run `git rm --cached backend/.env.test` to untrack. Provide backend/.env.test.example with placeholder values. Already added to .gitignore in a prior fix.

SOC2: CC6.2 **HIPAA:** — **CMMC:** CM.3.165 **ISO27001:** A.14.2.3

[F007] Missing rate limit on gift card redemption endpoint

What: POST /api/v1/gift-cards/{code}/redeem has no @limiter.limit decorator, allowing unrestricted gift card code enumeration.

Location: `backend/app/routers/extras.py:61–76`

Risk: Attackers can rapidly test gift card codes to find valid balances and drain them, especially if code format is predictable or sequential.

Fix: Add `@limiter.limit('5/minute')` decorator to the redeem endpoint. Consider also limiting the lookup endpoint.

SOC2: CC6.1, CC6.6 **HIPAA:** — **CMMC:** AC.2.006, SC.3.178 **ISO27001:** A.12.6.1

[F008] HSTS header configured on HTTP-only Caddy — header is ineffective

What: Caddyfile sets Strict-Transport-Security header but the server only listens on :80 (HTTP). HSTS is ignored by browsers on non-HTTPS responses.

Location: `caddy/Caddyfile`

Risk: The HSTS header provides no protection. If the deployment is ever HTTPS-enabled and HSTS was relied upon for security, it will have never been processed by browsers.

Fix: Either: (a) configure TLS in the Caddyfile to serve on :443 (Caddy auto-HTTPS handles this with a domain name), or (b) remove the HSTS header from the HTTP block and add it only when HTTPS is configured.

SOC2: CC6.1 **HIPAA:** 164.312(e)(2)(i) **CMMC:** SC.3.177 **ISO27001:** A.14.2.1

■■ INFO

[F009] Login endpoint distinguishes wrong password vs inactive account

What: auth.py returns HTTP 401 for bad credentials vs HTTP 403 for inactive account, allowing username enumeration of known-valid-but-inactive users.

Location: `backend/app/routers/auth.py:63–68`

Risk: Minor information disclosure: an attacker can determine that a username exists and has been deactivated (vs never existed). Low practical impact in a POS context.

Fix: Return generic HTTP 401 with identical message for both cases. Move the 'account inactive' distinction to a post-authentication step. Acceptable as-is given POS threat model.

SOC2: CC6.2 **HIPAA:** — **CMMC:** IA.1.076 **ISO27001:** A.9.4.2

[F010] CSP includes 'unsafe-inline' and 'unsafe-eval' in script-src

What: Both next.config.ts and caddy/Caddyfile include script-src 'unsafe-inline' 'unsafe-eval' in the Content-Security-Policy.

Location: `frontend/next.config.ts:12, caddy/Caddyfile`

Risk: These weaknesses are required by Next.js for inline scripts and hot module replacement. They reduce XSS mitigation effectiveness. Accepted risk for this stack.

Fix: Consider using CSP nonces (Next.js nonce support available in v13+) to eliminate 'unsafe-inline'. 'unsafe-eval' can be removed in production builds. Low priority given Next.js constraints.

SOC2: CC6.1 **HIPAA:** — **CMMC:** SC.3.181 **ISO27001:** A.8.20

[F011] No multi-factor authentication for admin or supervisor accounts

What: Authentication is username+password only with no second factor for privileged roles.

Location: `backend/app/routers/auth.py`

Risk: If admin credentials are compromised, an attacker gains full system access with no second barrier. Admin can void sales, modify settings, and view all reports.

Fix: Add optional TOTP (pyotp) as a second factor, required for admin/supervisor roles. Phased rollout acceptable.

SOC2: CC6.2 **HIPAA:** 164.308(a)(5)(ii)(C) **CMMC:** IA.3.083 **ISO27001:** A.9.4.2

Remediation Priority

Address open findings in this order:

#	Severity	Finding	Action	Effort
1	CRITICAL	F001	Add ownership check to GET /sessions/{id}: deny if cashier_id != current_user.id unless admin/supervisor	~1 hour
2	HIGH	F002	Add ownership check to GET /sales/{id} and void_sale: deny if cashier_id != current_user.id unless admin	~1 hour
3	HIGH	F003	Add scope check to void_sale: supervisor can only void sales by cashiers they supervise	~2 hours
4	MEDIUM	F004	Enforce non-default ADMIN_INITIAL_PASSWORD at startup in production; block boot if default	~30 min
5	MEDIUM	F005	Add tenant_id filtering to all service queries (prepares for multi-tenant)	~3 hours
6	LOW	F006	Run git rm --cached backend/.env.test to stop tracking the file	~5 min
7	LOW	F007	Add @limiter.limit('5/minute') to gift card redeem endpoint	~10 min
8	LOW	F008	Configure Caddy TLS or move HSTS to HTTPS-only block	~30 min

Confirmed Security Controls ■

- bcrypt password hashing (passlib) — no plain-text or MD5 passwords
- Pydantic validation on ALL API routes — no unvalidated user input reaches the DB
- SQLAlchemy parameterized queries throughout — zero SQL injection surface
- Sensitive fields excluded from API responses (password hashes, internal IDs)
- 3-tier RBAC enforced on every route: cashier / supervisor / admin
- JWT authentication required on all endpoints except /health and /api/v1/branding
- slowapi rate limiting: login 10/min, change-password 10/min
- Audit logs on ALL mutations: sales, purchases, returns, gift cards, users, settings
- JWT tokens expire in 60 minutes (reduced from 480 min in v1)
- Security headers in next.config.ts: CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy
- Security headers in Caddyfile: CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy
- SQLAlchemy echo=False always (was echo=not is_production in v1)
- FastAPI docs_url / redoc_url disabled in production
- Docker images: non-root users (appuser, nextjs), pinned base images
- Structured logging (structlog) — no credentials or PII in logs
- UUID primary keys — no sequential ID enumeration
- Soft deletes (deleted_at) — no hard data deletion
- .env excluded from git (.gitignore verified; git log shows no secret commits)
- TypeScript: zero type errors (npx tsc --noEmit confirmed)
- CORS origins locked (not wildcard) — enforced via CORS_ORIGINS env var

• Rebrand (feat/rebrand-kolekto-v1) introduced zero new security vulnerabilities

Audit History

Date	Version	C	H	M	L	Score	PDF
2026-05-07	v1	0	2	4	3	52	docs/security/2026-05-07-v1-security-audit.pdf
2026-05-08	v1	1	2	2	3	62	docs/security/2026-05-08-v1-security-audit.pdf